Classifying stars as either White Dwarfs or Subdwarf-B stars using Gaia data

by Thomas Oeser

Summary

Introduction

Neural Network classification

Part 1 : Downloading the first dataset
Part 2 : Importing pandas and reading the dataset
Part 3 : Creating the neural network
Part 4 : Cleaning the data
Part 5 : Investigating the suspiciously perfect model
Part 6 : Investigating the feature correlation
Part 7 : Re executing the neural network using only key features
Part 8 : Plotting our results
Part 9 : Using the neural network on a new dataset

Conclusion and future prospects

Introduction

Astrophysicists work with millions of stars, each of them unique in their own way. Despite their differences, stars follow tendencies and can be classified into groups. Properly classifying stars is an important part of an astrophysicist's job to better organize future work on them.

In this project specifically, we will be working on a code that will use Neural Networks and to classify stars in two groups : White Dwarfs stars and Subdwarf-B stars. For this, we will be using data from the Gaia telescope that provided us 4 datasets containing detailed information on tens of thousands of stars each. This project aims to aid my Master's Astrophysics research group in their classification methods.

For further information regarding the context of this project, it is recommended that the reader familiarizes themselves with the project presentation attached with this document. This presentation elaborates further on the importance of this work. But to make it short : White dwarfs and Subdwarf-B stars are 2 distinct types of stars with very different environments and behaviour, but they share some similarities that can make them tricky to classify manually.

Currently, the way that these stars have been distinguinshed and classified was using a somewhat arbitrary method called a "polynomial method". While this method was good in classifying many stars with a respectable accuracy, the idea of this project would be to use machine learning to hopefully get a higher accuracy. This program could then be used in future prospects to enhance classification and ease the work of this branch of astrophysics. For more information on how the polynomial method works, you may read the project presentation document.

For the classification methods, 2 methods in particular are going to be considered : neural networks and random forests. This is because the datasets contain a list of tens of thousands of stars with already correctly classified stars and a greatly structured set of numerical features.

A random forest classification method could aid us in interpretability, reduce overfitting and handling missing or noisy data. However, it may struggle to identify more complex patterns between features, where a neural network classification program might be better, which could potentially increase the accuracy. In the end, we might consider a hybrid model that will combine the strength of both classification methods.

For our project, we will be using the Google Colab platform and execute our program in a Google Colab Notebook. This is because Google Colab offers great CPU and GPU power that will be just enough for our purposes. The datasets we will be working with are csv files saved in Google drive and are listed in the project presentation document.

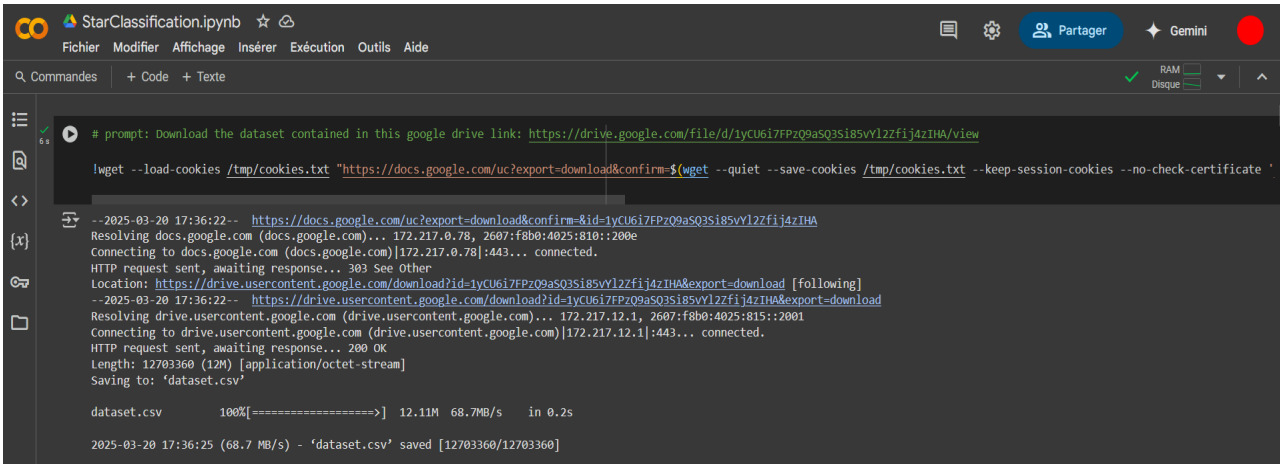
Neural Network Classification

We will start our project with a neural network classification. The first dataset we will be working on is the first dataset listed in the project presentation document. It is the one from this link :

https://drive.google.com/file/d/1yCU6i7FPzQ9aSQ3Si85vYl2Zfij4zIHA/view

Part 1 : Downloading the first dataset

We start with the first dataset and upload it onto our notebook. Since the datasets are from Google drive links, we can simply ask the Google AI provided in Google Colab to quickly write us a line of code to download the first dataset without the need of looking for for it :



Part 2 : Importing pandas and reading the dataset

We will need to have an overview of our dataset to know what we are working with, so we will need to print the head (the first 5 rows) of the dataset, as well as the number of stars we are working with. Preferrably, we want to know how many White Dwarfs we are working with and how many Subdwarfs. The stars are classified as either one type of star in the 'class' feature at the very far right end of the dataset. 'wd' stands for White Dwarf, and 'sdb' for Subdwarf-B. As a measure of security, we want to make sure that no other unique values are present in this feature. Some stars may have a misspelled 'class' feature or not even be either a WD or sdB star.

So we will be inputing the following :

```
# Imports
import pandas as pd

# Read dataset
df = pd.read_csv('dataset.csv')
print(df.head())

# Count the occurrences of each unique value in the 'class' column
class_counts = df['class'].value_counts()
class_counts
```

Executing this code gives us the head of our dataset and shows us that only 2 unique values exist in the 'class' feature, which is good for us because we won't have to correct data for this feature. We will be working with a total of 13 857 White Dwarf stars and 6 593 Subdwarf-B stars.

Part 3 : Creating the neural network

Let us first try a "shot in the dark" and see if we are lucky. We will create a neural network that will simply analyze all the features and try to figure out how to classify each star on its own, without us even cleaning the data. For this purpose, I have chosen the use of the TensorFlow deeplearning framework, because of its reputation to be easier of use.

In this part of the code, we will first be importing all the necessary libraries. (3.1)

Next, we want to filter in the stars that only have either 'wd' or 'sdb' as value for their 'class' feature. In our case, all stars have this. But even if this small step could be seen as redundant, it is still a good idea to implement it in case we might stumble upon a dataset with different values. (3.2)

Next, we define X as being all features except the target feature 'class', and we define y as just the feature 'class'. (3.3)

TensorFlow only works with numbers, so we will also need to convert the 'wd' and 'sdb' values to numerical values. (3.4)

Now we can split the dataset in training and testing sets. For this, we will be using the standard 0.8 to 0.2 ratio. (3.5)

Since we are creating a neural network, the next step is to normalize the data with a standard scaler to avoid that the machine considers a certain feature more important than another. We want our model to treat all features equally. (3.6)

Now we will create our neural network. We will start with a classic 3 layers with a moderate amount of neurons (64 and 32). (3.7)

Next we compile our model using the adam optimizer and the binary crossentropy loss function. Then we train the model with 10 epochs, a batch size of 32, and a classic 0.8 to 0.2 training data ratio. (3.8)

Lastly, we evaluate the model and make the predictions. (3.9)

Executing this bloc of code gives us the following result :

```
Epoch 1/10
/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `input_sha
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
409/409 ━━━━━━━━━━━━━━━━━━━━ 5s 5ms/step - accuracy: 0.6745 - loss: nan - val_accuracy: 0.6815 - val_loss: nan
Epoch 2/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.6788 - loss: nan - val_accuracy: 0.6815 - val_loss: nan
Epoch 3/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.6747 - loss: nan - val_accuracy: 0.6815 - val_loss: nan
Epoch 4/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.6775 - loss: nan - val_accuracy: 0.6815 - val_loss: nan
Epoch 5/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 3s 5ms/step - accuracy: 0.6756 - loss: nan - val_accuracy: 0.6815 - val_loss: nan
Epoch 6/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6767 - loss: nan - val_accuracy: 0.6815 - val_loss: nan
Epoch 7/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.6711 - loss: nan - val_accuracy: 0.6815 - val_loss: nan
Epoch 8/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6725 - loss: nan - val_accuracy: 0.6815 - val_loss: nan
Epoch 9/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6735 - loss: nan - val_accuracy: 0.6815 - val_loss: nan
Epoch 10/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6769 - loss: nan - val_accuracy: 0.6815 - val_loss: nan
128/128 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.6736 - loss: nan
Test accuracy: 0.6816626191139221
128/128 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
```

We can see that the model performed surprisingly passably with an accuracy of 68%, but as we can see, we have NaN values for the losses. This was to be expected, since we did not yet clean our data. The way that we will do it is by simply replacing all missing data by the mean value of each feature.

Part 4 : Cleaning the data

As previously mentioned, we will be cleaning the data by replacing missing or invalid values with the mean average value of each feature, except of course for the "class" feature that determines the type of star.

Re-executing the same neural network code with the cleaned data now gives us the following results :

```
Epoch 1/10
/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `input_shape`/`inpu
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
409/409 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.9289 - loss: 0.2352 - val_accuracy: 0.9954 - val_loss: 0.0223
Epoch 2/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 4s 6ms/step - accuracy: 0.9979 - loss: 0.0127 - val_accuracy: 0.9982 - val_loss: 0.0099
Epoch 3/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 5s 11ms/step - accuracy: 0.9997 - loss: 0.0036 - val_accuracy: 0.9985 - val_loss: 0.0080
Epoch 4/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - accuracy: 0.9998 - loss: 0.0019 - val_accuracy: 0.9982 - val_loss: 0.0089
Epoch 5/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9998 - loss: 0.0014 - val_accuracy: 0.9985 - val_loss: 0.0083
Epoch 6/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9987 - loss: 0.0038 - val_accuracy: 0.9994 - val_loss: 0.0049
Epoch 7/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.9999 - loss: 6.9987e-04 - val_accuracy: 0.9960 - val_loss: 0.0134
Epoch 8/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9966 - loss: 0.0096 - val_accuracy: 0.9988 - val_loss: 0.0058
Epoch 9/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.9999 - loss: 7.6556e-04 - val_accuracy: 0.9985 - val_loss: 0.0067
Epoch 10/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 1.0000 - loss: 2.5139e-04 - val_accuracy: 0.9991 - val_loss: 0.0064
128/128 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9988 - loss: 0.0038
Test accuracy: 0.9990220069885254
128/128 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
```

Now we get an insane accuracy of more than 99.9%. Furthermore, this accuracy differs only very little compared to the val_accuracy, and the loss and val_loss are both extremely low, indicating that there is barely any overfitting. But this is very suspicious.

Part 5 : Investigating the suspiciously perfect model

The numbers we just got seem too good to be true, so something might have gone wrong. To investigate this matter, we will adopt the following tactic :

First we will assign all the stars at random into 2 different groups. We will then train a neural network model on one of the two groups, and use that model to make predictions on the other.

First we assign the stars into 2 groups :

```
# Split the dataframe into two groups randomly
df_A, df_B = train_test_split(df, test_size=0.5, random_state=42)

# Count how many elements have either "wd" or "sdb" as value in their "class" feature for both groups
wd_sdb_count_A = df_A[df_A['class'].isin(['wd', 'sdb'])].shape[0]
wd_sdb_count_B = df_B[df_B['class'].isin(['wd', 'sdb'])].shape[0]

# Count how many of each "wd" and "sdb" are there in each group
wd_count_A = df_A[df_A['class'] == 'wd'].shape[0]
sdb_count_A = df_A[df_A['class'] == 'sdb'].shape[0]
wd_count_B = df_B[df_B['class'] == 'wd'].shape[0]
sdb_count_B = df_B[df_B['class'] == 'sdb'].shape[0]


print("Group A: Total elements with 'wd' or 'sdb' class:", wd_sdb_count_A)
print("Group A: Number of 'wd' elements:", wd_count_A)
print("Group A: Number of 'sdb' elements:", sdb_count_A)
print("-" * 20)
print("Group B: Total elements with 'wd' or 'sdb' class:", wd_sdb_count_B)
print("Group B: Number of 'wd' elements:", wd_count_B)
print("Group B: Number of 'sdb' elements:", sdb_count_B)
```

```
Group A: Total elements with 'wd' or 'sdb' class: 10225
Group A: Number of 'wd' elements: 6925
Group A: Number of 'sdb' elements: 3300
--------------------
Group B: Total elements with 'wd' or 'sdb' class: 10225
Group B: Number of 'wd' elements: 6932
Group B: Number of 'sdb' elements: 3293
```

Next, we clean the data using the mean average filling :

```
import pandas as pd
# Replace missing or invalid data with the mean value for each feature (except 'class')
for group_name, group_df in [('group_A', df_A), ('group_B', df_B)]:
  for column in group_df.columns:
      if column != 'class':
          group_df[column] = pd.to_numeric(group_df[column], errors='coerce')  # Convert to numeric, handle non-numeric as NaN
          group_df[column].fillna(group_df[column].mean(), inplace=True)  # Replace NaN with mean
```

Then we train a new model based on the data of "group_A" to make predictions on "group_B". Doing so yields us the following result :

```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `inp
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/10
256/256 ━━━━━━━━━━━━━━━━━━━━ 7s 9ms/step - accuracy: 0.8837 - loss: 0.3048 - val_accuracy: 0.9946 - val_loss: 0.0404
Epoch 2/10
256/256 ━━━━━━━━━━━━━━━━━━━━ 2s 9ms/step - accuracy: 0.9938 - loss: 0.0254 - val_accuracy: 0.9966 - val_loss: 0.0172
Epoch 3/10
256/256 ━━━━━━━━━━━━━━━━━━━━ 2s 9ms/step - accuracy: 0.9983 - loss: 0.0093 - val_accuracy: 0.9976 - val_loss: 0.0143
Epoch 4/10
256/256 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9997 - loss: 0.0038 - val_accuracy: 0.9976 - val_loss: 0.0145
Epoch 5/10
256/256 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9996 - loss: 0.0031 - val_accuracy: 0.9961 - val_loss: 0.0133
Epoch 6/10
256/256 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 1.0000 - loss: 0.0016 - val_accuracy: 0.9976 - val_loss: 0.0135
Epoch 7/10
256/256 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9999 - loss: 0.0011 - val_accuracy: 0.9976 - val_loss: 0.0120
Epoch 8/10
256/256 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 1.0000 - loss: 6.5542e-04 - val_accuracy: 0.9976 - val_loss: 0.0122
Epoch 9/10
256/256 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 1.0000 - loss: 7.3829e-04 - val_accuracy: 0.9976 - val_loss: 0.0141
Epoch 10/10
256/256 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 1.0000 - loss: 5.8464e-04 - val_accuracy: 0.9936 - val_loss: 0.0304
320/320 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9975 - loss: 0.0090
Group A accuracy: 0.996381402015686
320/320 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9941 - loss: 0.0270
Group B accuracy: 0.9946210384368896
```

We again stumble upon near perfect accuracy, which has merely "worsened" from 0.999 to 0.996 and 0.994 accuracy. With this test, we have proven that our model is not overfitted, as it could reach almost the same level of accuracy to predict the class of each star in one group using the data available in another group.

Part 6 : Investigating the feature correlation

Overfitting does not seem to be the problem here, but another possibility might explain this, perhaps one or more features are highly correlated with the 'class' feature of our dataset, and our model is using those few features as a "shortcut" to make its predictions, essentially not doing "real" work.

To test out for this, we could draw a correlation matrix, but given the very high number of different features in our dataset, this would create an unreadable image. So what we will do instead, is that we will simply print out the features that have the highest correlation with the 'class' feature.

Doing all this yields us the following results :

```
Top 10 features most correlated with 'class':
phot_rp_mean_mag                0.702950
phot_bp_mean_mag                0.700175
phot_g_mean_mag                 0.686968
phot_bp_mean_flux_over_error    0.600906
parallax                        0.598912
phot_g_mean_flux_over_error     0.565074
phot_rp_mean_flux_over_error    0.516071
parallax_error                  0.449333
pmdec_error                     0.440797
pmra_error                      0.427160
Name: class, dtype: float64
```

Now we need a little bit of context to explain these correlations, because they should not come to much as a surprise to an astrophysicist. We will need to explain a little astrophysics, but do not worry, we will not make it too complicated to understand.

The first 3 features with the highest correlation with the 'class' feature are *_mean_mag. In astrophysics, these are the mean magnitudes of a star in different wavelengths of light. "phot_rp" is the magnitude on the infrared light, "phot_bp" for the blue to ultraviolet light, and "phot_g" is the overall magnitude (or luminosity) of the star.

Since white dwarfs and subdwarfs are inherently different in these features, it is normal that our model uses these for its predictions and that they correlate pretty well. These features are an essential part for differentiating the stars.

Another feature that is very much correlated to the 'class' feature is the 'parallax', and this one is very interesting. To make it easy to understand, in astrophysics, the parallax refers to how much an observed star moves around in the night sky as the Earth orbits around the Sun : The more the star moves around over the course of the year, the closer it is to our solar system, and the less it moves, the further away it is from us.

The reason why the parallax is an interesting feature to analyze is because if there is a correlation between the distance of a star and whether it is a white dwarf or subdwarf, then it means that either one of them tends to be located closer to us than the other.

And sure enough, this is precisely what astrophysicists observe : White Dwarf stars are very small in size, barely a little bigger than the Earth, (1% the radius of our Sun), which causes them to be very dim. As a result, White Dwarfs can only be observed when they are close enough to our solar system.

Subdwarf-B stars on the other hand are a lot bigger by comparison, around 0.15 to 0.25 times the radius of our Sun, and hence a whole order of magnitude larger than White Dwarfs. This causes them to be much brighter in comparison, and hence they can be observed at a much greater distance.

And now we begin to understand how our neural network model actually thinks : The smaller the parallax of a star, the much further away it is from our solar system, and the more likely it is to be a Subdwarf-B star.

If the parallax is high, it means the star is much closer to us, and the model needs to check the observed magnitude, or luminosity of the star. If the luminosity is high, it will likely be a Subdwarf-B star, and if it is low, it will likely be a Subdwarf-B star.

If the model still can't decide by that point, it will check for the specific magnitudes of the star using the phot_rp and phto_bp features (ultraviolet and infrared light). Subdwarf-B tend to be more "blue" than White Dwarfs and should have higher phot_bp values and lower phot_rp values.

All the other top correlated features in this list (*_error) are just measurement imprecision errors. Naturally, those can't possibly have a correlation with the class feature. The only reason why they are showing as correlated is because these errors are themselves correlated with the magnitudes and parallax features.

By investigating the validity of the near perfect accuracy of our model, we have ruled out the possibility of overfitting, have tested the model on data it has not trained on, and no abnormal feature seems to be overly correlated with the class feature of our dataset. So it seems that very surprisingly, our model's accuracy is actually valid.

However, our work is still not very clean. At this point, our model is still using certain features that have obviously no correlation with the class of the star whatsoever, like the measurement errors discussed earlier and the 'source_id' feature, which is just the name of the star.

Furthermore, it is one thing to have numerical data, but it would be nicer to have a plot to visualize it all. If the plot shows data that is visually distributed at random, then our model is practically useless.

Part 7 : Re executing the neural network using only key features

To adress the cleanliness of our model, since we have found out the top features most related to the class feature of our stars, we can use those (the 3 magnitude features and the parallax feature) as the only features that our model should work on. With only these 4 features to work on for the new version of our model, we should hopefully get an accuracy of at least over 90%.

To adress the plot that we could draw, we could simply plot a "Herzsprung-Russel diagram" (which will be explained later) with colored dots representing both types of stars. If our model is good at classifying, we should expect two distinct clusters of points at specific locations on that diagram. If the classification is bad, we can expect a total mess.

Part 7.a : Rereading the dataset and dropping all unnecessary features

In this part we will do a simple "cleaning" of our dataset, using only the 4 aforementioned features. We then clean the data by replacing missing or invalid data with the mean average value of each feature (except for 'source_id' and 'class')

```
import pandas as pd

# Reread the dataset
df = pd.read_csv('dataset.csv')

# Select only the specified columns
selected_columns = ['source_id', 'phot_rp_mean_mag', 'phot_bp_mean_mag', 'phot_g_mean_mag', 'parallax', 'class']
df = df[selected_columns]

# Clean the data
for column in ['phot_rp_mean_mag', 'phot_bp_mean_mag', 'phot_g_mean_mag', 'parallax']:
    df[column] = pd.to_numeric(df[column], errors='coerce')
    df[column].fillna(df[column].mean(), inplace=True)

print(df.head())
```

```
             source_id  phot_rp_mean_mag  phot_bp_mean_mag  phot_g_mean_mag  \
0  2414485448212792064         17.088371         16.613232        16.769844
1  2773760665114010880         16.806425         16.385021        16.519869
2  2847977322031768832         14.426654         14.110446        14.219558
3  2313130542218587008         16.792944         16.311130        16.474659
4  2765454164004531328         13.777930         13.489800        13.589941

   parallax class
0  0.358506   sdb
1  0.309961   sdb
2  1.179264   sdb
3  0.366055   sdb
4  1.614939   sdb
```

Part 7.b : Create a neural network on the new dataframe

We then create a neural network based on these 4 features, and get the following result :

```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `input_shape`/`i
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 7s 9ms/step - accuracy: 0.9165 - loss: 0.2677 - val_accuracy: 0.9905 - val_loss: 0.0312
Epoch 2/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.9961 - loss: 0.0204 - val_accuracy: 0.9963 - val_loss: 0.0235
Epoch 3/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 3s 7ms/step - accuracy: 0.9959 - loss: 0.0187 - val_accuracy: 0.9966 - val_loss: 0.0176
Epoch 4/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9950 - loss: 0.0190 - val_accuracy: 0.9966 - val_loss: 0.0149
Epoch 5/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.9971 - loss: 0.0105 - val_accuracy: 0.9972 - val_loss: 0.0134
Epoch 6/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9981 - loss: 0.0102 - val_accuracy: 0.9976 - val_loss: 0.0111
Epoch 7/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9977 - loss: 0.0114 - val_accuracy: 0.9972 - val_loss: 0.0103
Epoch 8/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9984 - loss: 0.0074 - val_accuracy: 0.9979 - val_loss: 0.0098
Epoch 9/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9981 - loss: 0.0093 - val_accuracy: 0.9979 - val_loss: 0.0085
Epoch 10/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9989 - loss: 0.0061 - val_accuracy: 0.9976 - val_loss: 0.0075
128/128 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9985 - loss: 0.0045
Test accuracy: 0.9982885122299194
```

As we can see, we still have a very high accuracy with 99.8% and only 0.45% loss, which can still be quite surprising. So we will run a little experiment. We will create 2 other neural networks, but this time these models will run only on either the magnitude features or the parallax feature.

Part 7.c : Testing the model by dropping the parallax

In this part, we imply drop the parallax and let the neural network run only on the magnitude features of the stars. The results we get are the following :

```
  df[column].fillna(df[column].mean(), inplace=True)
/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `input_shape`/`i
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
409/409 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - accuracy: 0.8316 - loss: 0.4213 - val_accuracy: 0.8741 - val_loss: 0.3226
Epoch 2/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8709 - loss: 0.3245 - val_accuracy: 0.8802 - val_loss: 0.3144
Epoch 3/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8796 - loss: 0.3191 - val_accuracy: 0.8829 - val_loss: 0.3025
Epoch 4/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8836 - loss: 0.3026 - val_accuracy: 0.8857 - val_loss: 0.2927
Epoch 5/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8850 - loss: 0.2945 - val_accuracy: 0.8878 - val_loss: 0.2836
Epoch 6/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8820 - loss: 0.2893 - val_accuracy: 0.8897 - val_loss: 0.2777
Epoch 7/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.8889 - loss: 0.2767 - val_accuracy: 0.8894 - val_loss: 0.2732
Epoch 8/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.8895 - loss: 0.2737 - val_accuracy: 0.8909 - val_loss: 0.2724
Epoch 9/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8864 - loss: 0.2772 - val_accuracy: 0.8933 - val_loss: 0.2653
Epoch 10/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.8862 - loss: 0.2779 - val_accuracy: 0.8943 - val_loss: 0.2638
128/128 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.8976 - loss: 0.2624
Test accuracy: 0.8970659971237183
```

Now we get an accuracy with values that look much more familiar to what we are generally used to from everything that we have learned in our exercises and homework, with "only" 89% accuracy for a 26% loss. This means that not only is this model less accurate, it is also less confident in its predictions.

And it makes sense with what we have learned about how our model works : by only having the magnitude to work with, the model can predict that stars that are very bright must certainly be Subdwarf-B stars. But stars that are dim could either be White Dwarfs or very distant Subdwarf-B stars. Without the parallax as a reference, the model is less accurate.

Part 7.d: Testing the model using only the parallax

Let us see what happens if we test for only the parallax. Using the same neural network but this time using only the parallax, we get the following result :

```
  df[column].fillna(df[column].mean(), inplace=True)
/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `input_shape`/`i
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
409/409 ━━━━━━━━━━━━━━━━━━━━ 3s 4ms/step - accuracy: 0.8486 - loss: 0.3683 - val_accuracy: 0.9603 - val_loss: 0.1345
Epoch 2/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9570 - loss: 0.1397 - val_accuracy: 0.9618 - val_loss: 0.1329
Epoch 3/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9603 - loss: 0.1335 - val_accuracy: 0.9609 - val_loss: 0.1306
Epoch 4/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9593 - loss: 0.1364 - val_accuracy: 0.9621 - val_loss: 0.1301
Epoch 5/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9606 - loss: 0.1310 - val_accuracy: 0.9615 - val_loss: 0.1295
Epoch 6/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9564 - loss: 0.1428 - val_accuracy: 0.9615 - val_loss: 0.1308
Epoch 7/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9596 - loss: 0.1373 - val_accuracy: 0.9612 - val_loss: 0.1284
Epoch 8/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.9593 - loss: 0.1350 - val_accuracy: 0.9618 - val_loss: 0.1296
Epoch 9/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9630 - loss: 0.1251 - val_accuracy: 0.9603 - val_loss: 0.1286
Epoch 10/10
409/409 ━━━━━━━━━━━━━━━━━━━━ 2s 2ms/step - accuracy: 0.9598 - loss: 0.1284 - val_accuracy: 0.9606 - val_loss: 0.1285
128/128 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - accuracy: 0.9582 - loss: 0.1381
Test accuracy: 0.958679735660553
```

Here, we get 95% accuracy and 13% loss. There is a very strong correlation between parallax and the type of star observed. Again, it makes sense : Higher parallaxes suggest closer stars, and therefore they are more likely to be White Dwarfs, since we tend to see those only in our close stellar neighbourhood.

However, relying only the parallax feature is not scientific. With better observation tools we will be able to detect White Dwarfs that are much further away from us, and this will skew the accuracy of a model that relies on this feature alone. Furthermore, such a model wouldn't be able a distinguish White Dwarfs from Subdwarfs if the dataset contained Subdwarfs located exclusively in our close stellar neighbourhood without information on the luminosity. Therefore, combining both the magnitude features and parallax features is crucial to make the distinction.

This little experiment is also further proof that the near perfect accuracy of our initial model is valid and not an overfitting.

Part 8 : Plotting our results

With all the numerical data gathered, and the confirmation that our model is accurate but also that its accuracy is valid, we will now draw 2 plots to illustrate our results and have a better overview.

The first plot that we will draw is a Herzsprung-Russel diagram. This diagram was explained in the project presentation, but we will do a quick recap. A Herzsprung-Russel diagram (or HRD for short, also sometimes called Color Magnitude Diagram, or CMD) plots the stars such that the y axis represents the magnitude, or luminosity of the star, and the x axis represents the color magnitude of the star (or surface temperature).

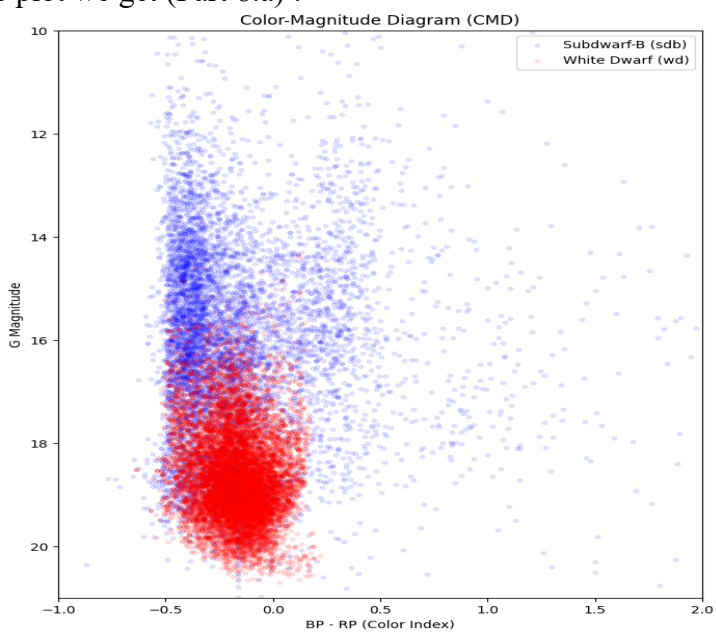
The higher a point is on that diagram, the more luminous it is. The more it goes to the left of the diagram, the more blueish it is in color. The more it goes to the right, the more reddish it becomes instead.

The y axis is called the G magnitude. It is defined in our dataset by the 'phot_g_mean_mag' feature. Careful though : a lower G magnitude means a more luminous star, while a greater G magnitude means a dimmer star.

The x axis is the color index, it is defined by 'phot_bp_mean_mag' - 'phot_rp_mean_mag'

Subdwarf-B type stars are bluer and brighter than White Dwarfs. As such, in such a diagram, they should tend to be higher up and more to the left than White Dwarf stars. Since they are bluer in color, it is only fitting that we will color them blue in this diagram, and red for White Dwarfs. Since we have a big dataset, we will also add a little transparency to minimize points covering others.

And here is the plot we get (Part 8.a) :

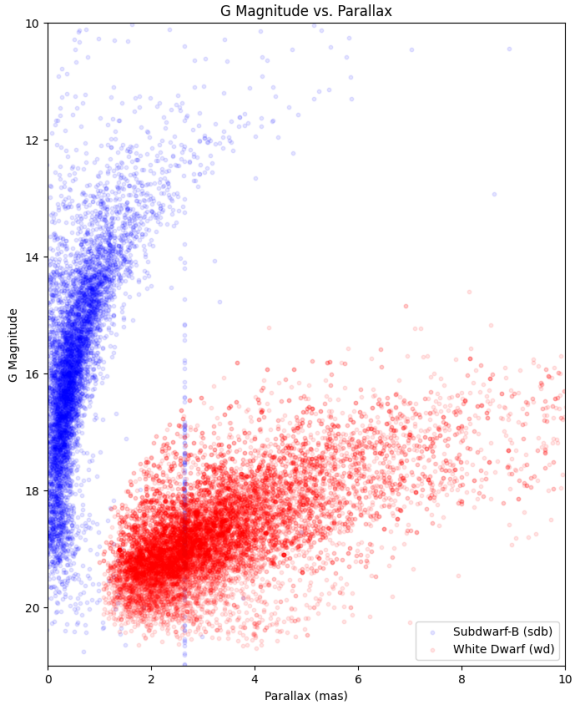


Just as we have predicted, the Subdwarf-B stars are mainly concentrated at the top left of the White Dwarfs. So our model was able to more or less correctly arrange our stars in a HRD, which is further proof as to why it did so well.

Note that the "wall" at BP-RP=-0.5 is normal. Points located further to the left are outliers that have some sort of mistake in their values.

Now, we will plot the magnitude against the parallax of the star (Part 8.b). Remember : a lower parallax indicates stars located further away, which increases the chances of it being a Subdwarf-B star. A higher parallax indicated it's closer and more likely a White Dwarf. As such, we should expect another cluster of blue points located at the top left side of a cluster of red points.

Plotting the magnitude against the parallax gives us the following diagram :

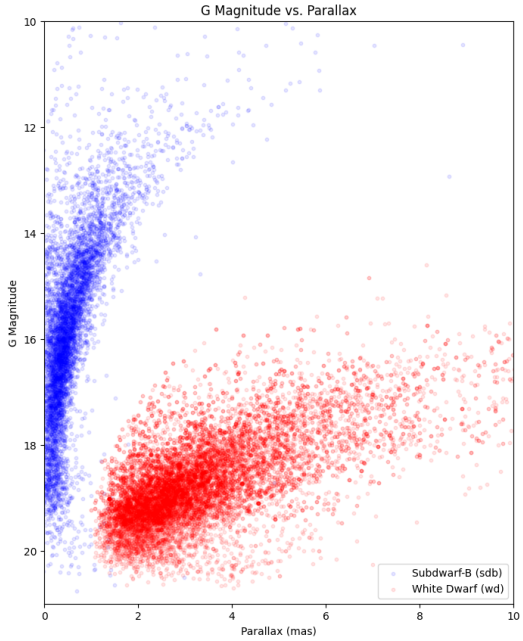


And now it becomes much clearer as to why our model did so well and why it is almost flawless in its predictions. In this plot we can clearly distinguish the two types of stars, as the two clusters don't intermingle.

Interestingly, we can observe a vertical blue line in our plot. This is probably due to us choosing to fill missing or invalid values with the mean average. A significant amount of Subdwarf-B type stars may have faulty data in their parallax value, and as such they are given the mean average, which seems to be around 2.5.

This line will probably disappear if we dropped the stars with faulty parallax values instead of filling them in, which we do in part 8.c of the notebook. For good measure, we also count how many stars ar ebeing dropped. They are 171 in total, all Subdwarf-B stars.

And here is the plot we get as a result :



Part 9 : Using the neural network on a new dataset

Now that we have confirmed that our model works well with the first dataset of around 20,000 stars, we can now try to use it on the second dataset containing over 60,000 stars. Unlike the first dataset, this one does not have a 'class' feature, and as such it is impossible to test if our model is correct in a prediction it made for a star. But it's alright, since the goal of our work is to try and identify the stars of the second dataset in the first place.

The second dataset is provided in the below Google drive link :
https://drive.google.com/file/d/1iV2rFZK9OJZWqzO2I9SP2PgQhlZWC1eE/view

Part 9.a: Downloading the dataset and dropping all unnecessary features

Just like with the first dataset, we can simply ask the AI to download the dataset for us, and ask it to name it df2 instead of df.

```
# prompt: Download the dataset from the following Google Drive link and print its dimensions: https://drive.google.com/file/d/1iV2rFZK9OJZWqzO2I9
# Call the dataframe as df2

import pandas as pd
!wget --no-check-certificate 'https://drive.google.com/uc?export=download&id=1iV2rFZK9OJZWqzO2I9SP2PgQhlZWC1eE' -O dataset.csv
df2 = pd.read_csv('dataset.csv')
df2.shape
```

```
--2025-03-28 01:00:06--  https://drive.google.com/uc?export=download&id=1iV2rFZK9OJZWqzO2I9SP2PgQhlZWC1eE
Resolving drive.google.com (drive.google.com)... 172.253.115.102, 172.253.115.101, 172.253.115.139, ...
Connecting to drive.google.com (drive.google.com)|172.253.115.102|:443... connected.
HTTP request sent, awaiting response... 303 See Other
Location: https://drive.usercontent.google.com/download?id=1iV2rFZK9OJZWqzO2I9SP2PgQhlZWC1eE&export=download [following]
--2025-03-28 01:00:06--  https://drive.usercontent.google.com/download?id=1iV2rFZK9OJZWqzO2I9SP2PgQhlZWC1eE&export=download
Resolving drive.usercontent.google.com (drive.usercontent.google.com)... 142.250.31.132, 2607:f8b0:4004:c0b::84
Connecting to drive.usercontent.google.com (drive.usercontent.google.com)|142.250.31.132|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 66725130 (64M) [application/octet-stream]
Saving to: ‘dataset.csv’

dataset.csv         100%[===================>]  63.63M  60.3MB/s    in 1.1s

2025-03-28 01:00:11 (60.3 MB/s) - ‘dataset.csv’ saved [66725130/66725130]

(61585, 153)
```

This time however, we will directly drop all unnecessary features :

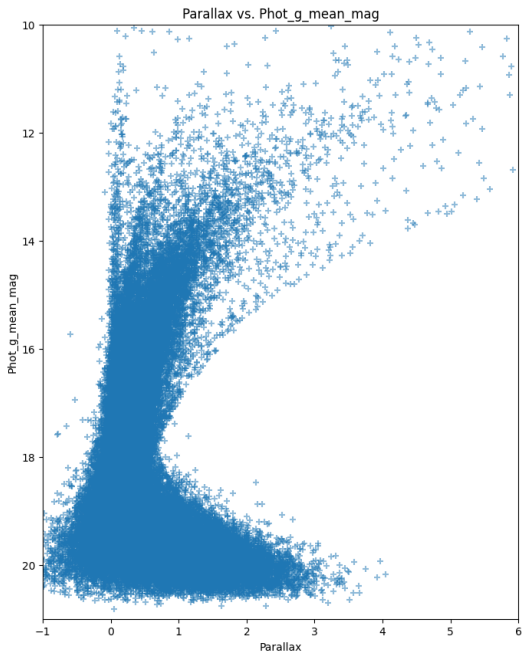
```python
# Keep only the specified columns in df2
df2 = df2[['source_id', 'phot_rp_mean_mag', 'phot_bp_mean_mag', 'phot_g_mean_mag', 'parallax']]

# Print the head of the new dataframe
print(df2.head())
```

```
             source_id  phot_rp_mean_mag  phot_bp_mean_mag  phot_g_mean_mag  \
0   604474251290699776         14.859774         14.465498        14.597474
1   619153556155078272         14.664076         14.692213        14.705356
2   614650192750888320         16.526846         17.007193        16.964804
3  1978338700510656256         12.696702         12.412711        12.511989
4  5624975872798794752         16.769178         16.724384        16.747410

   parallax
0  0.861644
1  1.142478
2  0.752423
3  1.745146
4  0.470509
```

Since we had such good results with plotting magnitude vs the parallax in our first dataset, we can try to do the same here. And this is the plot we get :



The stars seem to form the shape of a boot. From the knowledge we have gathered with our work so far, we can easily identidy the two main clusters of points : The front of the "foot" should represent White Dwarf type stars, and the leg, all the way down to the heel, should represent the Subdwarf-B type stars.

However, we stumble onto two complications : the first is that unlike in our first dataset, the two clusters mingle with each other, making a clear delimitation more complicated, so we will need to add an uncertainty factor to our model.

The second complication is that some stars have a "negative" parallax, which is physically impossible, since stars can't be "slower than a standstill".

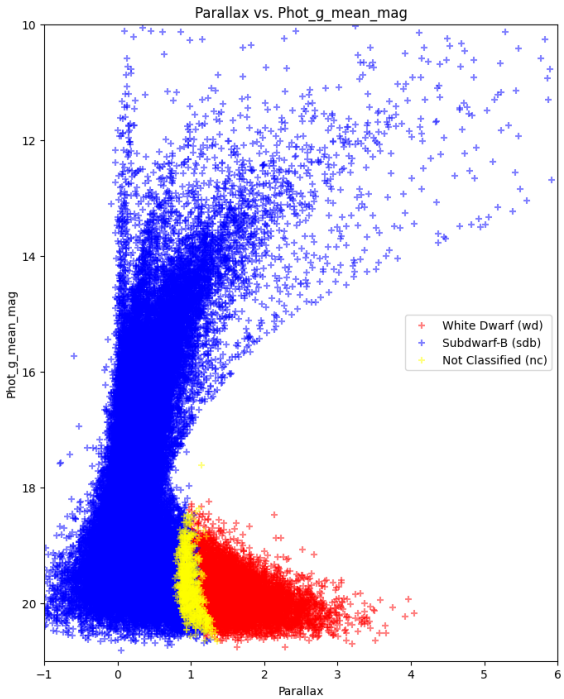
Part 9.c : Using the neural network to classify the stars, (with uncertainty threshold)

To adress the first problem, we will add an uncertainty feature into our model. Simply put, if our model is uncertain about whether a star should be classified as either a WD or SDb star, it will classify it as "nc" instead, for "Not Classified". We do this by designing a threshold. The way our model works, is that it grades a star from 0 to 1. The model then classifies the star depending on whether the grade is closer to 0 or 1. By defining a threshold in between (for example 0.4 to 0.6), we can add a 3rd option for our model.

```
  df2_prepared[col].fillna(df2_prepared[col].mean(), inplace=True)
1925/1925 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step
             source_id  phot_rp_mean_mag  phot_bp_mean_mag  phot_g_mean_mag  \
0   604474251290699776         14.859774         14.465498        14.597474
1   619153556155078272         14.664076         14.692213        14.705356
2   614650192750888320         16.526846         17.007193        16.964804
3  1978338700510656256         12.696702         12.412711        12.511989
4  5624975872798794752         16.769178         16.724384        16.747410

   parallax class
0  0.861644   sdb
1  1.142478   sdb
2  0.752423   sdb
3  1.745146   sdb
4  0.470509   sdb
Number of elements with 'wd' class in df2: 8769
Number of elements with 'sdb' class in df2: 52004
Number of elements with 'nc' class in df2: 812
```

In our code, we could filter out 812 stars that our model was uncertain about. Now if we plot the data with 'nc' stars colored in yellow, we get the following plot :



Our model succesfully classified the stars. The threshold could be thiner (for example 0.45 to 0.55), but a bigger threshold allows us to have a little more leeway in how certain our model can make correct predictions. Besides, "only" around 800 stars were classified as uncertain, from over 60,000 to classify. The model already filtered out a good chunk of stars and saved up already a lot of work to do.

Since our research group would like to work on those 800 stars, a good idea would be to create a downloadable txt file that lists all of them :

```python
import pandas as pd
from google.colab import files

# Create a list to store the lines for the text file
file_lines = []

# Iterate through the 'nc' stars in df2
for index, row in df2[df2['class'] == 'nc'].iterrows():
    rank = index + 1  # Calculate the rank
    source_id = row['source_id']
    prediction = predictions[index][0]  # Access the prediction value
    classification = 'wd' if prediction < 0.5 else 'sdb' # Determine the classification
    file_lines.append(f"{rank} {source_id} {prediction:.4f} {classification}") #format the output

# Create the text file
with open('nc_stars.txt', 'w') as f:
  f.write('\n'.join(file_lines))

# Download the file
files.download('nc_stars.txt')
```

nc_stars.txt

Fichier Modifier Affichage

```
13377 5924517955854476288 0.5103 sdb
13399 2919100223811056896 0.5054 sdb
13405 2919166744267639424 0.4567 wd
13413 2920755263690567296 0.5030 sdb
13416 3425368875352532864 0.5107 sdb
13480 4493834216069411968 0.4392 wd
13559 2845559457306926080 0.4153 wd
13577 2841588433624162560 0.5783 sdb
13726 6003157359643193472 0.5063 sdb
13758 2923082826726391296 0.4045 wd
13774 2923184398407954816 0.4401 wd
13782 2073715974086852096 0.4494 wd
13796 2923885715029925376 0.4486 wd
13814 2922817981866267136 0.4718 wd
13836 4197710445951662848 0.4545 wd
13853 3485005252171738752 0.5657 sdb
13859 5304835195323741824 0.4331 wd
```

Ln 1, Col 1 29 533 caractères 100% Unix (LF) UTF-8

This list contains all the essential data : the rank of the star in the dataset, its source_id name, the value predicted by the model, and whether it's more likely a WD or SDb star.

Part 9.d: Reclassifying stars with negative parallax

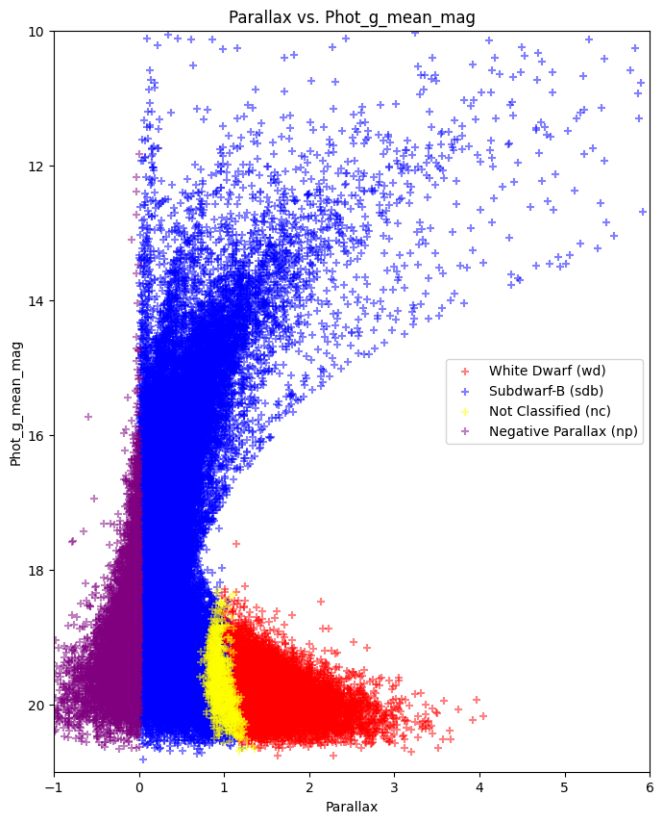
Now we can work on the stars with negative parallax. This isn't too complicated, we just create a new class type of stars classified 'np' for 'negative parallax and assign all stars with negative parallax into it. We'll color them in purple in our plot.

```python
# Change the 'class' value for elements with negative parallax
df2.loc[df2['parallax'] < 0, 'class'] = 'np'

# Print the head to verify
print(df2.head())
```

```
             source_id  phot_rp_mean_mag  phot_bp_mean_mag  phot_g_mean_mag  \
0  604474251290699776         14.859774         14.465498        14.597474
1  619153556155078272         14.664076         14.692213        14.705356
2  614650192750888320         16.526846         17.007193        16.964804
3  1978338700510656256        12.696702         12.412711        12.511989
4  5624975872798794752        16.769178         16.724384        16.747410

   parallax class
0  0.861644   sdb
1  1.142478   sdb
2  0.752423   sdb
3  1.745146   sdb
4  0.470509   sdb
```



Conclusion and prospects

Our Neural Network model had an excellent accuracy to classify stars as either White Dwarf stars or Subdarf-B stars. We investigated its near 100% accuracy and determined that it was actually pretty valid, because both types of stars could clearly be separated in an amplitude vs parallax plot.

For future prospects in my master research group, I will try to create a random forests model to classify the stars. This is done in order to have a more interpretable model with feature impotances scores. Such details will be useful for my research group. The neural network we just created will still be very useful though, because random forests perform better in smaller datasets.

While they shouldn't struggle with datasets containing tens of thousands of stars like the ones we worked on, we plan to use such models of datasets that contain tens of millions of stars. An idea would be to have a hybrid model where a neural network would make a first 'rough' classification, eliminating the majority of stars and then feeding the new dataframe to a random forest model.

Our model has another limitation : It is only trained to recognize White Dwarfs and Subdwarf-B type stars. On bigger datasets of stars that contain ther types (like Red Giants or Main Sequence stars that are similar to our Sun), we may need to update the model with new thresholds so that it classifies stars as yet another, different class, either by feeding it data of stars of another class, or by simply classifying them as unclassifiable.

Some errata :

In the original project presentation, I had mentioned that we would be working on 4 datasets, but we worked on only two of them instead. My mistake was that one of the remaining two datasets was a duplicate that was already contained in the first dataset we had worked on, and the other dataset was given to me by my supervisor by mistake, and did not contain relevant data.

Another mistake I made was to call the g-mean magnitude vs bp-rp magnitude diagrams a "Herzsprung-Russel Diagram" (or HRD). This is technically wrong but still an acceptable name. The actual name of such a plot is a "Color Magnitude Diagram" (CMD). The difference is subtle, but a HRD plots g mean magnitude against the surface temperature of the star, while a CMD plots the g mean magnitude on the apparent color of the star. The reason why even astrophysicists use these names interchangeably despite this subtle difference is that they don't make a difference between temperature and color : When an astrophysicist knows a star's color, they automatically know its temperature, and vice-versa.